

程序调试及应用手册



南京凌鸥创芯电子有限公司

程序调试及应用手册

版本：v1.0.0

© 2024, 版权归凌鸥创芯所有

机密文件，未经许可不得扩散

目录

- 1. 文档说明..... 1
- 2. 系统框图..... 2
 - 2.1 电流环 2
 - 2.2 速度环 2
- 3. 基准值检查计算..... 4
 - 3.1 电流基准值..... 4
 - 3.2 电压基准值..... 4
 - 3.3 频率/角度基准值..... 5
 - 3.4 基准值及参数设置校验..... 5
- 4. 硬件保护..... 7
 - 4.1 过流保护检查..... 7
 - 4.2 过压保护检查..... 7
- 5. 工程设置..... 8
 - 5.1 配置工程 8
 - 5.2 控制模式配置..... 9
 - 5.3 控制参数配置..... 10
 - 5.4 命令给定值配置 10
 - 5.5 驱动函数检查..... 12
- 6. 上电检查..... 13
- 7. 驱动输出检查 16
 - 7.1 固定输出模式..... 16
 - 7.2 VF 输出模式..... 17
- 8. 速度环闭环调试..... 20
 - 8.1 无感启动参数配置..... 20
 - 8.2 电流环参数配置 21
 - 8.3 速度环参数配置 23
 - 8.4 电流环调试及参数查看 24
 - 8.5 速度环调试及参数查看 25
 - 8.6 观测器参数查看 25
- 附录..... 27

程序调试及应用手册

R.1 数据格式27

 R.1.1 标幺定义27

 R.1.1.1 电流标幺27

 R.1.1.2 电压标幺28

 R.1.1.3 速度标幺28

 R.1.1.4 角度标幺28

 R.1.2 标幺函数28

 R.1.3 数据表示30

R.2 观测器介绍32

R.3 参数整定33

 R.3.1 电流环 PI 参数33

 R.3.2 速度环 PI 参数34

1. 文档说明

本文档仅是对电机控制软件简明介绍，基于新平台软件 V2_1_0a 版本呢，用于工程师进行程序应用及调式指南，便于工程师进行快速调试及问题定位分析。

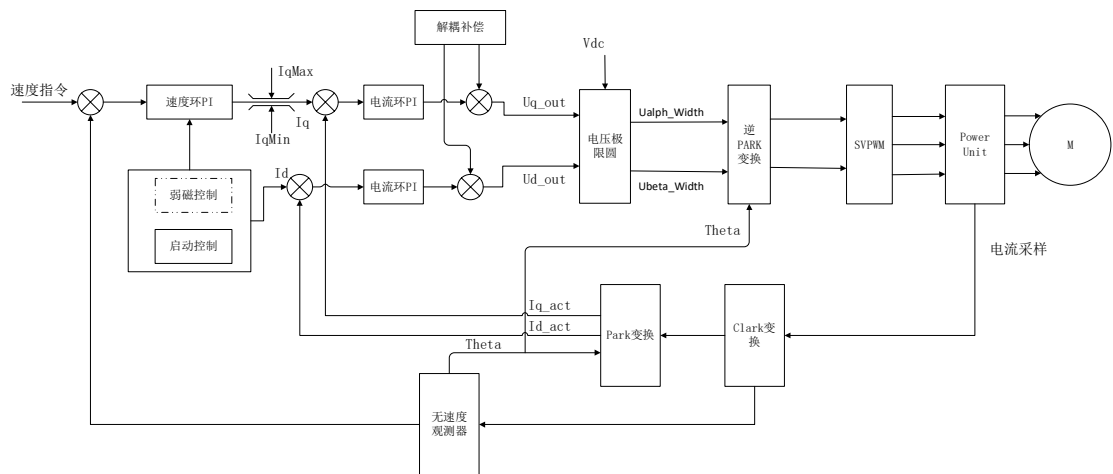
实际内容若有迭代更新，以最新软件为主。

文件说明	新平台软件程序应用及调试手册
版本	V1.0.0
日期	2024.12.17
适用 MCU	LKS32MC08x/LKS32MC03x/LKS32MC45x / LKS32MC07x
编制	Lith



2. 系统框图

无感控制系统控制框图如下：



框图中用到数据变量的数据格式等详见附件文档。

2.1 电流环

电流环：用于转子磁场 FOC 定向后的 dq 轴电流控制，并带有解耦补偿控制模块，可以通过宏配置参数进行配置。

电压极限圆模块：根据最大设定调制电压进行输出电压的脉宽计算，通过逆 park 单元输出 α/β 轴电压占空比。

SVPWM 模块：根据采用的电流采样方式（单/双/三电阻采样）进行脉宽调制，输出脉宽比较值，通过 MCU 的 MCPWM 模块输出驱动功率器件的脉宽信号。调制输出的脉宽信号受电流采样方式、功率器件的死区时间以及采样保持时间的影响。

无速度观测器：通过施加到电机电压以及实际采集的电机相电流信息，利用电机模型进行无速度观测，输出电机电角度及电机频率/速度信息，实现电机的 FOC 磁场定向及速度环控制。

2.2 速度环

速度环：用于速度环 PI 控制，输出电流指令 I_{qRef} 。

当采用功率环控制时，功能类似。

弱磁控制：速度增高时，电机控制需要的电压高于母线输出的最大调制电压，为进一步控制电机，需要启用弱磁控制功能，输出弱磁控制电流给定值 I_{dRef} ，实现恒功率控制。

启动控制：无感控制时，通过启动控制实现电机的无感启动等功能。

3. 基准值检查计算

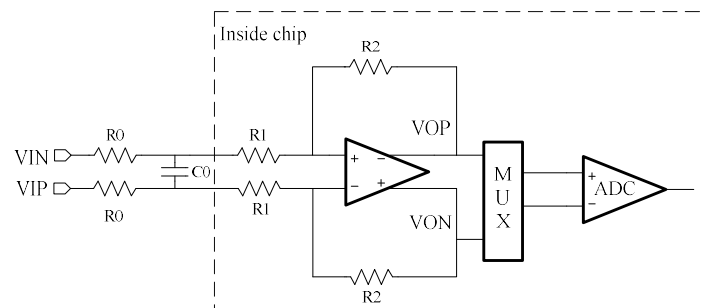
根据当前应用的原理图进行硬件配置完成后，检查配置确保硬件配置无误后进行代码调试。

首选进行电流、电压基准值计算，以及频率基准值设置，确保所有应用到的设置参数在合理的范围内。

当前程序中采用 **AD 量程** 作为标么计算的基准值，数据采用 IQ15 的数据格式。

因此需要检查基准值数值，不合适的采样范围（电压/电流基准值），或者应用参数配置，会导致程序或者电机运行异常，损坏硬件。

3.1 电流基准值



最终的放大倍数为 $\text{Gain} = R2 / (R1 + R0)$

因此电流值基准值为 $\text{Imax} = 3.6 / (\text{Gain} * \text{Rshunt})$

程序中的放大倍数配置及采样电阻配置，通过配置参数可以计算出电流值基准值为 Imax 。

```
/* -----Hardware Parameter----- */
#define ADC_SUPPLY_VOLTAGE_M0      (3.6)      /* 单位: V ADC基准电压, 3.6或者2.4, */
#define AMPLIFICATION_GAIN_M0      (17.5439) /* 运放放大倍数 */
#define RSHUNT_M0                   (0.05)    /* 单位: Ω 采样电阻阻值 */
```

$$\begin{aligned} \text{Imax} &= \text{ADC_SUPPLY_VOLTAGE} / (\text{AMPLIFICATION_GAIN} * \text{RSHUNT}) \\ &= 3.6 / (17.5439 * 0.05) = 4.10399056 \text{ A} \end{aligned}$$

3.2 电压基准值

根据硬件电路的母线采样电路，计算母线采样分压系数 $\text{AMPLIFICATION_GAIN}$

```
#define VOLTAGE_SHUNT_RATIO_M0      (1.0 / (20.0 * 2 + 1.0)) /* 母线电压分压比 */
#define BEMF_SHUNT_RATIO_M0        (1.0 / (20.0 * 2 + 1.0)) /* 反电势电压分压比 */
```

则母线电压基准值 $\text{Umax} = \text{ADC_SUPPLY_VOLTAGE} / \text{AMPLIFICATION_GAIN}$

3.3 频率/角度基准值

频率基准值 U_MAX_FREQ_Mx 在程序中设置

```
/* -----Rated Parameter----- */
#define U_MAX_FREQ_M0 (1333.0) /* 单位:Hz, 电机最高运行电频率 */
```

角度基准值默认： 角度作为无符号数时，0~360° 对应 0 ~ 65536

3.4 基准值及参数设置校验

在进行硬件保护检查确保无误后，可以上电链接电路板，打开 LKSScope 后添加观察变量查看电压电流基准值变量

gS_MotorObsM[0].mFluxObsVar.motorVoltageBase	14760	14760	14760	14760	
gS_MotorObsM[0].mFluxObsVar.motorCurrentBase	410	410	410	410	

上图中电压、电流变量为 app 格式数据

电压基准值 gS_MotorObsM[ID].mFluxObsVar.motorVoltageBase = 14760

对应 $U_{max} = 147.6V$

电压基准值 gS_MotorObsM[ID].mFluxObsVar.motorCurrentBase = 410

对应 $I_{max} = 4.1A$

检查程序配置的所有电压、电流变量是否设置合理，理论上所有设置值必须小于基准值。

例如，电流设置部分

电机启动电流

```
#define ID_START_M0 (1.0) /* 单位:A, D轴电流设定值 */
#define IQ_START_M0 (1.0) /* 单位:A, Q轴电流设定值 */
```

速度环输出电流限制

```
#define IQMAX_M0 (2.0) /* 单位:A, 速度环输出最大值 */
#define IQMIN_M0 (-2.0) /* 单位:A, 速度环输出最小值 */
```

以及软件过流电流设置值

```
#define I_PH_OVERCURRENT_FAULT_M0 (3.5) /* 单位:A 软件过流检测设定值 */
```

都需要小于 I_{max} 。

设置超出时代表超出了 AD 采样的最大量程，程序运行异常。需要较大的电

流值时，需要通过改变运算放大倍数（减小），或者减小采样电阻阻值，进而通过增大 AD 采样范围对应的实际电流值满足实际需要。

电压相关的量程及参数配置，进行同样的检查，例如

```
///  
/* 过欠压检测参数 */  
#define U_OVERTVOLTAGE_FAULT_M0      (30)      /* 单位：V 过压检测设定值 */  
#define U_OVERTVOLTAGE_RECOVER_M0    (26)      /* 单位：V 过压恢复设定值 */  
#define U_UNDERVOLTAGE_FAULT_M0      (12)      /* 单位：V 欠压检测设定值 */  
#define U_UNDERVOLTAGE_RECOVER_M0    (18)      /* 单位：V 欠压恢复设定值 */
```

设置的电压值需在电压基准值（电压量程）范围之内。

4. 硬件保护

检查硬件电路的保护功能，确保在调试过程中防止硬件配置错误或者参数设置等问题导致的硬件损坏。

需要检查的内容包括过流保护、过压保护，以及特殊应用的特殊保护，确保不会由于配置出错、程序异常等原因损坏硬件，确保不会损坏设备、甚至导致人身危险。

相关保护点根据功率器件、电容等硬件中最弱项进行设置，确保硬件通过的电压、承受的电压等在器件允许范围内。

4.1 过流保护检查

根据原理图，进行电流保护电路的保护电流及保护时间计算检查，确保硬件过流保护电路的有效性。

确保过流保护点在功率器件的 SOA 工作区域之内，防止控制异常时功率器件通过大的电流导致功率器件损坏。

```
/* 过流检测参数 */  
#define I_PH_HARD_OC_FAULT_CMP_VOLT_M0      (400)      /* 单位: mV 硬件过流保护比较电压值 */  
#define I_PH_OVERCURRENT_FAULT_M0           (3.5)      /* 单位: A 软件过流检测设定值 */  
#define I_PH_OVERCURRENT_FAULT_TIMES_M0     (5)        /* 单位: 软件过流检测次数 */
```

需要根据当前采用的过流保护电路进行分析，进行硬件配置、设置合适的保护数值。

4.2 过压保护检查

电压保护检查，检查过压保护设置参数是否合理

```
/* 过欠压检测参数 */  
#define U_OVERVOLTAGE_FAULT_M0              (30)        /* 单位: V 过压检测设定值 */  
#define U_OVERVOLTAGE_RECOVER_M0            (26)        /* 单位: V 过压恢复设定值 */  
#define U_UNDERVOLTAGE_FAULT_M0             (12)        /* 单位: V 欠压检测设定值 */  
#define U_UNDERVOLTAGE_RECOVER_M0           (18)        /* 单位: V 欠压恢复设定值 */
```

针对带预驱的芯片，过压时需要设置单点单次触发过压保护，防止瞬时的过压导致预驱或 LDO 的损坏。

5. 工程设置

5.1 配置工程

根据 MCU 类型或者倍频后的主频设置 MCU 的 clock，project_config.h。

```
#define MCU_MCLK_USED (MCU_MCLK_96M) // 当前MCU主频，根据MCU修改
```

默认

```
#define MCU_MCLK_192M (192LL)
#define MCU_MCLK_96M (96LL)
#define MCU_MCLK_48M (48LL)
```

采用其他主频时自行设置。

配置电流采样方式：

```
#define EPWM0_CURRENT_SAMPLE_TYPE (CURRENT_SAMPLE_1SHUNT) // 单电机平台电流采样方式配置
```

根据当前选用的 MCPWM 配置相关参数

```
#if (PLANTFORM_DRV_MODULE_NUM == 1) //驱动单元模块个数为1

// 驱动单元0配置
#define DRV0_PWM_ID (PLANTFORM_EPWM0) // 选用EPWM0
#define DRV0_PARA_ID (0) // 选用参数组0，B
#define DRV0_CUR_SAMP_TYPE (EPWM0_CURRENT_SAMPLE_TYPE)
```

MCU 支持多组 MCPWM 时，需要根据选用的 MCPWM 进行响应的配置。

单电机默认选用 DRV0。

功能配置：

```
#define QUICK_CURRENTLOOP_FUNCTION (FUNCTION_ON) // 快速电流环模式，缩短电流环执行时间
#define OBSERVER_PLL_CHANGE_ENABLE (FUNCTION_ON) // 观测器PLL调整使能
#define VF_START_FUNCTION_ENABLE (FUNCTION_ON) // VF启动使能
#if (VF_START_FUNCTION_ENABLE == FUNCTION_ON)
#define VECTOR_VF_COMP_ENABLE (FUNCTION_ON) // VF启动使能时，VECTOR VF补偿自动使能
#else
#define VECTOR_VF_COMP_ENABLE (FUNCTION_OFF) // VECTOR VF补偿使能
#endif
#define POWER_CALC_CALIB_FUNCTION_ENABLE (FUNCTION_ON) // 功率校正使能  $y=a2 * x^2 + a1 * x + a0$ 
#define RTI_FUNCTION FUNCTION_ON /* RTI 调试功能 */
```

默认启用 VF 功能，用于开环 VF 控制以及电流环解耦补偿。

下载参数配置：

目前程序默认配置参数放在最后的 1K 或 2K Flash 空间，针对不同 FLASH 存储容量的芯片，需要根据芯片进行 FLASH 配置。

```
#define ADDR_DOWNLOAD_WRITE_LOCATION (0x20001000)
#define ADDR_DOWNLOAD_READ_LOCATION (0x20001001)
#define ADDR_PARA_LOCATION (0xFE00)
#define ADDR_PARA_START_LOCATION (ADDR_PARA_LOCATION + sizeof(STR_DrvCfgGlobal))
```

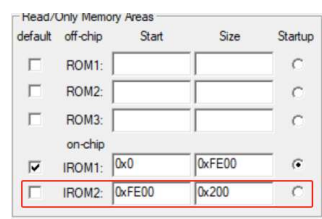
程序中指定存储空间，

```
// 配置参数通过编译器伪指令放置在指定的FLASH空间
const STR_DrvCfgPara tS_DrvCfgPara[MAX_CFG_PARA_ID] __attribute__((at(ADDR_PARA_START_LOCATION))) = DRV_CFG_PARA_TAB_H(0);

// 全局配置参数通过编译器伪指令放置在指定的FLASH空间
const STR_DrvCfgGlobal tS_DrvCfgGlobal __attribute__((at(ADDR_PARA_LOCATION))) = DRV_CFG_GLOB;

/*****
#endif (PARA_DOWNLOAD_ENBALE == FUNCTION_ON)
u8 gs_bDownReqCmd __attribute__((at(ADDR_DOWNLOAD_WRITE_LOCATION))) = 0;
u8 gs_bDownAckStatus __attribute__((at(ADDR_DOWNLOAD_READ_LOCATION))) = 0;
u8 gs_bDownloadStatus = 0;
u16 gs_timeCnt;
#endif
*****/
```

Keil 编译器指定 FLASH 分区，



FLASH 宏定义及 Keil 编译器指定分区必须匹配。目前用于 LKSMotor 配置数据下载空间，用 LKSMotor 功能时这些配置参数不允许修改。

不需要 LKSMotor 功能时，可以去掉伪指令__attribute__指定 FLASH 存放空间，并修改 Keil 编译器分区设置，由编译器自动分配 FLASH 空间。

5.2 控制模式配置

主要用于配置 PWM 频率，调制系数，控制环路、预充电、顺逆风等功能。

MC_GlobalDef_Mx.h

配置功率器件参数，参阅功率器件手册进行合适的死区配置及采样参数配置

```
/* -----PWM 频率及死区定义----- */
#define MCU_MCLK_M0 (MCU_MCLK_USED) /* PWM模块运行主频 */
#define PWM_MCLK_M0 ((u32)MCU_MCLK_M0) /* PWM模块运行主频 */
#define PWM_PRSC_M0 (0) /* PWM模块运行预分频器 */
#define PWM_FREQ_M0 ((u16)16000) /* PWM斩波频率 */

#define CURRENT_SAMPLE_TYPE_M0 (DRV0_CUR_SAMP_TYPE) /* 电流采样方式选择 */
#define ROTOR_SENSOR_TYPE_M0 (ROTOR_SENSORLESS) /* 电机位置传感器类型选择 */

#define SVFWM_TYPE_M0 (0) /* 0 --- 7seg 1--- 5seg */
#define PWM_USED_ID_M0 (DRV0_PWM_ID)
#define OPA_SELECT_M0 (0) /* 运算放大倍数 选择 */

#define DEADTIME_NS_M0 ((u16)1200) /* 死区时间500--1000 */
#define DEADTIME_M0 (u16) (((unsigned long long)PWM_MCLK_M0 * (unsigned long long)DEADTIME_NS_M0) / 1000000)

#define ADC_COV_TIME_M0 (200) /* 预留的ADC转换时间，单位：500--200ns */
#define SAMP_STABLE_TIME_SHUNT_M0 (1500) /* 1000--1800单电阻采样，信号稳定时间设置 */
```

```
/* -----IPD 转子初始位置检测 脉冲注入时间设置----- */
#define SEEK_POSITION_STATUS_M0 (TRUE) // 初始位置检测状态 TRUE 为使能, 1

/* -----顺逆风检测是否是能----- */
#define DIR_CHECK_STATUS_M0 (FALSE) // 顺逆风检测状态 TRUE 为使能, 1

/* -----align 对相是否是能----- */
#define ALLIGN_STATUS_M0 (TRUE) // ALLIGN对相状态 TRUE 为使能, 1

/*-----PWM使能控制----- */
#define PWM_ENABLE_STOP_M0 (FALSE) // PWM状态使能, TRUE = 使能
#define STOP_MODE_M0 (0) // 停止方式, 0 = 关闭MOE, 1 = 保持MOE

#define FAULT_AUTO_CLR_M0 (TRUE) // 故障自动清除使能
#define FAULT_CLR_PERIOD_M0 (2000) // 故障清除周期 ms
```

参阅程序或者手册，根据应用进行相应的配置

5.3 控制参数配置

MC_Parameter_Mx.h

电机参数：

```
#define U_MOTOR_PP_M0 (5.0) /* 电机极对数 */
#define U_MOTOR_RS_M0 (0.444273) /* 单位: Ω 电机相电阻 */
#define U_MOTOR_LD_M0 (234.4485) /* 单位: uH 电机d轴电感 */
#define U_MOTOR_LQ_M0 (343.893) /* 单位: uH 电机q轴电感 */

/* 电机磁链常数 计算公式: Vpp/2/sqrt(3)/(2*PI)/f, 其中Vpp为电压峰值, f为电频率
   此参数仅影响顺逆风启动的速度检测, 角度估算不使用些参数 */
#define U_MOTOR_FLUX_CONST_M0 (0.0174687451)
```

控制参数等参阅相关手册及程序，按照应用进行相应设置及配置。

5.4 命令给定值配置

在 6_UserAppLayer 文件夹下设置相关命令值，默认采用 UA_DEBUG 配置，

```
#ifdef UA_DEBUG
volatile bool runM0UaCmd = FALSE;
bool runM0UaCmdAuto = FALSE;
volatile s32 runM0SpdSetpoint = 50*100;
s16 runM0Period = 0;

```

runM0UaCmd 为启动信号，TRUE 时运行电机，FALSE 时停止电机，停止方式取决于配置的停止方式。

采用速度环时，runM0SpdSetpoint 为频率设定值

采用其他控制模式时，查看下面的函数及控制模式配置等

程序调试及应用手册

```

/*****
函数名称:      void updateUASetpoint(pSTR_UACtrlProc pObj)
功能描述:      单电机UA指令处理函数
输入参数:      pSTR_UACtrlProc pObj
输出参数:      无
返回值:        无
其它说明:      用户可根据需要添加相应的处理函数或者处理方式
修改日期      版本号      修改人      修改内容
-----
2022/8/19      V1.0      Tonghua Li      创建
2022/3/29      V1.1      Tonghua Li      更新
*****/
void updateUASetpoint(pSTR_UACtrlProc pObj)
{
    PSTR_DrvCfgPara pDrvCfgPara = pObj->mg_pCtrlObj->m_pDrvCfgPara;

    #if (DRV0_CLOSE_LOOP == SPEED_LOOP)          //速度环

    #if (DRV0_POWER_LIMIT_STATUS == TRUE)          //速度环限功率
        pObj->m_wSetpoint      = pDrvCfgPara->mS_FBSpdLoop.m_wPowerLmtSpdValue;
        pObj->m_wPowerSetpoint = pDrvCfgPara->mS_FBSpdLoop.m_wPowerLmtValue;
    #endif

```


5.5 驱动函数检查

最新版本程序中，为减少函数调用时间，对单电机的驱动函数等进行了优化，更改为直接调用或着 inline 函数调用的方式。

如 SVPWM 函数和 PWM 更新函数，需要在调用的地方直接修改函数。

```
// dq电流环 PI 计算
RegMotorCurrentDQInline(tS_pFocElememt);

// SVPWM 调制
#if 0
tS_pMotorFoc->m_pHandle->Svpwm(t_bObjId,&tS_pFocElememt->mStatVolAB);
#else
    #if (EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_2SHUNT)
        SVPWM_2Shunt_Enl(t_bObjId,&tS_pFocElememt->mStatVolAB);
    #elif (EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_1SHUNT)
        SVPWM_1Shunt_Mode0(t_bObjId,&tS_pFocElememt->mStatVolAB);
    #else
        SVPWM_3Shunt(t_bObjId,&tS_pFocElememt->mStatVolAB);
    #endif
#endif

// PWM 占空比更新
HAL_DRV0_MCPWM_RegUpdate(tS_pMotorFoc->pMdToHd);
```

双电机版本，仍然通过修改注册函数实现 FOC 和 HAL 驱动函数的手动更改。

```
//V2_1_0以后版本单电机时，直接在FOC_Model函数中调用SVPWM函数
#if (SVPWM_TYPE_M0 == 0)
#if (EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_2SHUNT)
#define Motor_FocMethod_M0 \
{ /* FOC Methods */ \
    SVPWM_2Shunt_Enl, /* SVPWM调制 */ \
}
#elif (EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_1SHUNT)
#define Motor_FocMethod_M0 \
{ /* FOC Methods */ \
    SVPWM_1Shunt, /* SVPWM调制 */ \
}
#else
#define Motor_FocMethod_M0 \
{ /* FOC Methods */ \
    SVPWM_3Shunt, /* SVPWM调制 */ \
}
#endif
#endif

#define Motor_HALDrv_EPWM0 \
{ /* DRV handle */ \
    AcquireCurSamplingResultsM0, /* 读取电流AD采样 */ \
    AcquireVdcSamplingResultsM0, /* 读取母线AD采样 */ \
    AcquireNTCSamplingResultsM0, /* 读取NTC AD采样 */ \
    AcquireBmefSamplingResultsM0, /* 读取BMEF AD采样 */ \
    ChangeADChanelCFG0, /* 切换AD采样通道 */ \
    MCPWM0_RegUpdate, /* PWM加载 */ \
    EPWM0_OutPut, /* PWM输出使能 */ \
    EPWM0_Charge, /* PWM预充电 */ \
    InitEPWM0ChargeEnd, /* PWM预充电初始化 */ \
    GetEPWM0_breakInStatus, /* 读取 BreakIn 状态 */ \
    ClrEPWM0_breakInStatus /* 清除 BreakIn 状态 */ \
}
```

应用时注意单电机和双电机版本的区别，再正确地放进行程序修改等。

6. 上电检查

根据电气原理图进行硬件及 MCU 底层驱动配置，再根据实际应用进行工程配置，并设置电机参数及合适的控制参数后，编译程序下载后进行初次上电。

上电后通过 LKSScope 检查电压、电流基准值，并再次查看设置参数。

同时通过 LKSScope 查看 UA 相关状态：

UA					
变量描述	当前值	最小值	最大值	平均值	
runMOSpdSetpoint	5000	5000	5000	5000	
sS_UACtrlProc.[0].m_eUASysState	1-E_UA_STOP		1	1	
sS_UACtrlProc.[0].m_StruCmD.m_bRunFlag	0	0	0	0	
sS_UACtrlProc.[0].m_bLoopMode	1	1	1	1	
sS_UACtrlProc.[0].m_bStopMode	0	0	0	0	
sS_UACtrlProc.[0].m_wPowerSetpoint	0	0	0	0	>
sS_UACtrlProc.[0].m_wTqSetpoint	1	1	1	1	>
sS_UACtrlProc.[0].m_wSetpoint	50	50	50	50	>
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mUaToMa.wFreqCmd	50	50	50	50	>
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mUaToMa.wTorqCmd	1	1	1	1	>
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mUaToMa.wPowerCmd	0	0	0	0	>
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.wVdcDec	23.88	23.74	24.03	23.8701	>
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.wCurDec	0.007	0	0.016	0.00520226	>
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.wVolDec	0	0	0	0	>
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.wSpeedDec	0	0	0	0	>
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.uFault	0	0	0	0	
sS_MACtrProc.[0].m_wPower001W	0	0	0	0	>
DebugTmax	655	655	655	655	>
DebugT1	647	568	647	606.736	

正常时：

- 1) UA 状态为 E_UA_STOP
- 2) gS_PhyObjInstanceM[0].mGlobalDatPackage.mMaToUa.wVdcDec
为当前母线电压数值，APP 格式。示例的数据已转化为 U_{sr} 数据，对应 23.88 V，实际母线供电电压为 24V
- 3) gS_PhyObjInstanceM[0].mGlobalDatPackage.mMaToUa.uFault
存在故障时，显示故障数值，对应程序查找详细故障信息。

检查中断时序是否满足设定

1) 程序中定义编译宏，编入 1ms 计数变量 Debug1MsTimesRecord

```
#define TIME_TEST_DEFINED (1)
#if TIME_TEST_DEFINED
extern s16 Debug1MsTimes;
extern s16 Debug1MsTimesRecord;
#endif

函数名称: void Task_Scheduler(void)
功能描述: 按时间片任务调度函数
输入参数: 无
输出参数: 无
返回值: 无
其它说明:
修改日期 版本号 修改人 修改内容
2020/8/5 V1.0 Howlet Li 创建

void Task_Scheduler(void)
{
```

2) 中断函数打开编译宏

```
#define TIME_TEST_DEFINED
#ifdef TIME_TEST_DEFINED
volatile s16 DebugT0;
volatile s16 DebugT1;
volatile s16 DebugT2;
volatile s16 DebugTmax;
volatile s16 Debug1MsTimes;
volatile s16 Debug1MsTimesRecord;
#endif

#if (EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_1SHUNT)
void MCPWM_IRQHandler(void)
{
    //PWM0 时基0中断标志
    MCPWM_IF = BIT1 | BIT0; //T1事件|T0事件 注意，触发了ADC0 通道采样
    ADC0_IF |= BIT1 | BIT0;

    #ifdef TIME_TEST_DEFINED
    Debug1MsTimes++;
    HALL_CNT = 0;
    #endif

    AdcEocIsrDRV0();

    #ifdef TIME_TEST_DEFINED
    DebugT1 = HALL_CNT;
    DebugT2 = DebugT1;
    DebugT2 = DebugT1 - DebugT0;
    DebugTmax = DebugTmax < DebugT2 ? DebugT2 : DebugTmax;
    #endif
}

函数名称: void TIMER0_IRQHandler(void)
功能描述: TIMER0中断处理函数
输入参数: 无
输出参数: 无
返回值: 无
其它说明:
修改日期 版本号 修改人 修改内容
2020/8/5 V1.0 Howlet Li 创建

void TIMER0_IRQHandler(void)
{
    /* 时基500us */
    TIMER_IF |= TIMER_IF_ZERO;

    qS_TaskScheduler.bTimeCnt1ms++;
    qS_TaskScheduler.nTimeCnt10ms ++;
    qS_TaskScheduler.nTimeCnt500ms++;
}
```

3) LKSScope 查看计数值

DebugT1	568	568	647	610.224	
DebugTmax	655	655	655	655	x*1
Debug1MsTimes	5	0	18	8.01293	
Debug1MsTimesRecord	15	14	18	15.9698	

Debug1MsTimesRecord 数值平均值为 fPWM / KHz 附近数值

DebugTmax 为电流环中断执行时间， $\text{DebugTmax} / \text{主频}$ 为执行时间 us

gS_TaskScheduler.bTimeCnt1ms	0	0	1	0.454545
gS_TaskScheduler.nTimeCnt10ms	11	0	19	8.71429
gS_TaskScheduler.nTimeCnt500ms	91	91	989	549.753

正常计数跳动

如有异常，根据异常信息查看对应的硬件配置及参数配置，根据异常存在原因分析解决问题。

7. 驱动输出检查

根据当前采用的功率器件的驱动极性以及驱动方式进行 MCPWM 配置后，参阅功率器件手册进行合适的死区配置及采样参数配置，然后进行 MCPWM 输出驱动信号检查。

调试前需要确保过流保护正常工作，并且电流保护值再功率器件允许的工作范围之内，防止配置不正确时导致功率器件直通、损坏器件。

可以采用具有快速过流保护的电源系统进行电源供电，电路板工作异常时电源系统快速切断供电，防止器件损坏。

7.1 固定输出模式

可以采用强制输出占空比的方式检查 PWM 驱动信号及电平是否满足需求，编入下图的 debug 调试代码：

```
int main(void)
{
    if(!ConfigData_check())           /* 检查配置参数 */
    {
        while(1);                   /* 配置参数或库不支持，进入死循环状态 */
    }
    __disable_irq();                 /* 关闭中断 中断总开关 */
    Hardware_init();                 /* 硬件初始化 */
    sys_init();                      /* 系统初始化 */
    __enable_irq();                  /* 使能中断 */

    for(;;)
    {
        //DebugPWM_OutputFunction(); // debug 固定PWM 输出波形，用于测试硬件
        // 也可以采用vr测试
        // 测试时确保硬件过流等起作用

        Task_Scheduler(); /* 任务调度函数，根据时间分片处理 */
    }
}
```

检查 PWM 寄存器配置，设置合适的脉宽输出

```
*****
函数名称: void DebugPWM_OutputFunction(void)
功能描述: PWM输出功能调试 输出25%占空比
输入参数: 无
输出参数: 无
返回值: 无
其它说明:
修改日期 版本号 修改人 修改内容
-----
2017/11/5 V1.0 Howlet Li 创建
*****
void DebugPWM_OutputFunction(void)
{
    MCPWM_TH00 = (s16)(~(PWM_PERIOD_M0 >> 2));
    MCPWM_TH01 = (PWM_PERIOD_M0 >> 2);
    MCPWM_TH10 = (s16)(~(PWM_PERIOD_M0 >> 2));
    MCPWM_TH11 = (PWM_PERIOD_M0 >> 2);
    MCPWM_TH20 = (s16)(~(PWM_PERIOD_M0 >> 2));
    MCPWM_TH21 = (PWM_PERIOD_M0 >> 2);

    PWMOutputs(ENABLE);
    while(1)
    {
    }
```

然后示波器测量 PWM 信号，查看电平及死区时间设置是否满足要求，以及 PWM 周期/频率是否为配置的斩波频率。

7.2 VF 输出模式

电路板工作正常时，可以采用 VF 模式运行电机，进行相关的硬件信号测量。

采用电流开环，VF 控制的方式可以进行驱动信号、死区及采样电流的检查。

设置电流开闭环频率设定值以及启动电流

```
/*-----开环参数-----*/
#define OPEN_ANGLE_TAG_FREQ_M0      (8.0)      /* 单位: Hz 开环拖动最终频率 */
#define FREQ_ACC_M0                  (20.0)      /* 单位: Hz/s 开环拖动频率加速调整值 */
#define FREQ_DEC_M0                  (20.0)      /* 单位: Hz/s 开环拖动频率减速调整值 */

#define MATCH_TIME_M0                (5)         /* 估算和给定电流匹配次数, 当前未用, */

#define MIN_RUN_FREQ_M0              (30.0)      /* 单位: Hz 最低速度 */
#define CLOSE2OPEN_FREQ_M0          (2.0)      /* 单位: Hz 切换为开环拖动频率 */
#define CURRENTLOOP_CLOSE_FREQ_M0   (0.0)      /* 单位: Hz 电流闭环频率 */

#define ID_START_M0                  (1.0)      /* 单位: A, D轴电流设定值 */
#define IQ_START_M0                  (1.0)      /* 单位: A, Q轴电流设定值 */
```

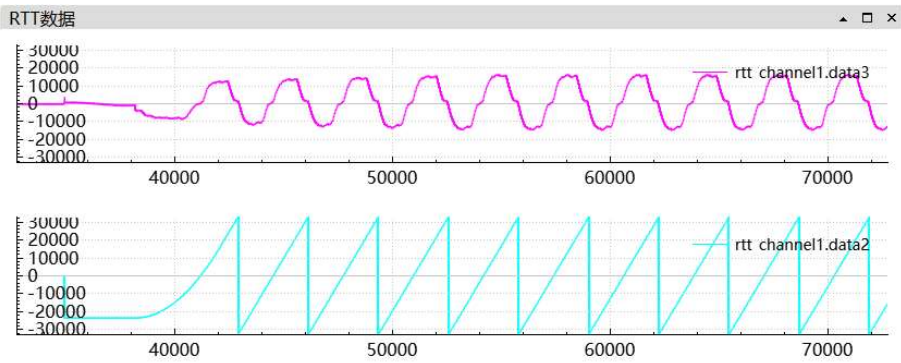
采用速度环模式时，如果速度命令设定值 runM0SpdSetpoint 小于上面的 CURRENTLOOP_CLOSE_FREQ_M0 设定值时，运行过程为开环 VF 模式。

此时可以进行死区时间检查、驱动信号电平测量，并且可以实时读取电流采样，检查电流采样电路设置是否正确、是否存在干扰。

正常下 LKSScope RTT 抓取变量

data3 ----- VF 启动运行的电流采样值，当前配置为单电阻采样

data2 ----- VF 启动运行的电角度



UA 状态，发送指令 5Hz，小于最小频率，所以 UA 状态为 STOP

UA						
变量描述	当前值	最小值	最大值	平均值	计数	
runM0UaCmd	1	0	1	0.948296		
runM0SpdSetpoint	500	500	500	500		
sS_UACtrlProc.[0].m_eUASysState	1-E_UA_STOP	1	1	1		

- 1) 母线电压 23.84V
- 2) 电机相电流峰值 1.772A,

程序调试及应用手册

- 3) 施加到电机的电压 1.8V,
- 4) 电机转速 3.62Hz (一阶滤波器截止影响)
- 5) 不存在报警

gS_PhyObjInstanceM.[0].mGlobalDatPackage.mUaToMa.wFreqCmd	5	5	5	5	x*0.01
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mUaToMa.wTorqCmd	1	1	1	1	x*0.001
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mUaToMa.wPowerCmd	0	0	0	0	x*0.01
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.wVdcDec	23.84	23.76	24.13	23.9382	x*0.01
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.wCurDec	1.772	0.001	2.326	1.82189	x*0.001
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.wVolDec	1.8	0.83	1.8	1.79362	x*0.01
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.wSpeedDec	3.62	0	3.62	3.39988	x*0.01
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.uFault	0	0	0	0	

MA 状态

sS_MACtrProc.[0].m_nLoopMode	1	1	1	1
sS_MACtrProc.[0].m_eSysState	8-E_DRIVER_RUN	8	8	8
sS_MACtrProc.[0].m_SMActrlBit.m_bMC_RunFlg	1	1	1	1
sS_MACtrProc.[0].mg_UFault.R	0	0	0	0
sS_MACtrProc.[0].m_nDCur_Set	7992	7992	7992	7992
sS_MACtrProc.[0].m_nDCur_Reference	7992	7992	7992	7992
sS_MACtrProc.[0].m_nQCur_Set	15984	15984	15984	15984
sS_MACtrProc.[0].m_nQCur_Reference	15984	15984	15984	15984
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToMd.eMotorStatus	6-E_MOTOR_SYN	6	6	6
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToMd.wMotorSpeedRef	122	122	122	122
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToMd.mStatCurDQCm.n...	7992	7992	7992	7992
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToMd.mStatCurDQCm.n...	15984	15984	15984	15984
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToMd.ePwmCmd	1-E_PWM_ON	1	1	1

- 1) 状态为 E_DRIVER_RUN
- 2) 无故障 sS_MACtrProc.[0].mg_UFault.R
- 3) D 轴电流给定 sS_MACtrProc.[0].m_nDCur_Reference = 7992
实际为 $7992/32767 * 4.10 = 1.0A$
- 4) D 轴电流给定 sS_MACtrProc.[0].m_nQCur_Reference = 15984
实际为 $15984/32767 * 4.10 = 2.0A$
- 5) 发送给 MD 命令为 E_MOTOR_SYN、E_PWM_ON
- 6) 发送给 MD 速度指令为 122, 即 $122/32767 * 1333 = 4.963Hz$

MD 层变量查看

```
/******//全局变量定义区
STR MotorFoc      gS_FocObjM[MAX_DRV_MODULE_USED]; // FOC 控制对象定义
STR MotorFocElement gS_FocElementObjM[MAX_DRV_MODULE_USED]; // FOC 变量定义
u8                g_buMdFWMONDelay[MAX_DRV_MODULE_USED]; // PWM ON delay 计数
```

数据结构中包含所有的 FOC 变量信息

```
typedef struct{
    //s16 nUdc;
    u8 m_bObj_ID;
    STR_VectorUVW mStatCurrUVW; /*定子电流UVW */
    STR_VectorAB mStatCurrAB; /*定子电流AB */
    STR_VectorDQ mStatCurDQCmd; //DQ电流指令
    STR_VectorDQ mStatCurDQ; /*定子电流dq */
    STR_MotorFocAngle mMotorAngle; /*角度结构体 */
    STR_MotorFocSpeed mMotorSpeed; /*速度结构体 */
    STR_PIRegulator mAcrD; /*d轴电流调节 */
    STR_PIRegulator mAcrQ; /*q轴电流调节 */
    STR_VectorDQ mAcrOutDQ; /*电流环输出 */
    STR_VectorAB mStatVolAB; /*定子电压AB */
    STR_TrigComponents mSinCosFocAngle; //FOC角度正弦余弦值
    STR_FluxObsGainCoefDef mFluxObsGain;
}STR_MotorFocElement,*PSTR_MotorFocElement;
```

测试中存在异常时，进行硬件及软件配置检查。

8. 速度闭环调试

硬件工作正常后进行正常的电机控制参数等配置，实现需要的控制功能。

正常调式时首先进行无感参数配置，设定开环闭环等工作区间。

然后配置电流环参数，检查电流采样顺序等，确保 FOC 定向准确，然后检查电流闭环控制是否正常。

电流环电流负反馈工作正常后，配置速度环控制参数进行速度环调试。

调试时进行参数查看，根据需要调整控制参数等，满足系统的动态性能和调速要求。

选择速度环运行模式

```
/*-----环路选择-----*/
#define CLOSE_LOOP_M0 (SPEED_LOOP) // 环路选择
```

8.1 无感启动参数配置

任务执行在 AD 采样中断或 MCPWM 中断中，执行频率取决于 PWM 载波频率设置。

可以配置观测器执行标志，进行降频执行。

MD 层代码

```
if(tS_pMotorFoc->bObserWork)
{
    tS_pMotorFoc->bObserWork = FALSE;
    CalcDcVoltPerUnit(t_bObjId,tS_pMotorFoc->pH
}
else
{
    tS_pMotorFoc->bObserWork = TRUE;
}
```

gS_FocObjM[].bObserWork 观测器执行标志，TRUE 执行观测器。

通过控制该标志位即可控制中断中观测器执行的频率，目前为观测器执行频率为 PWM 载波的一半。

设置无感控制参数：

```
/*-----开环参数-----*/
#define OPEN_ANGLE_TAG_FREQ_M0 (15.0) /* 单位: Hz 开环拖动最终频率 */
#define FREQ_ACC_M0 (20.0) /* 单位: Hz/s 开环拖动频率加速调整值 */
#define FREQ_DEC_M0 (20.0) /* 单位: Hz/s 开环拖动频率减速调整值 */

#define MATCH_TIME_M0 (5) /* 估算和给定电流匹配次数, 当前未用, 预留 */

#define MIN_RUN_FREQ_M0 (30.0) /* 单位: Hz 最低速度 */
#define CLOSE2OPEN_FREQ_M0 (2.0) /* 单位: Hz 切换为开环拖动频率 */
#define CURRENTLOOP_CLOSE_FREQ_M0 (10.0) /* 单位: Hz 电流闭环频率 */

/*-----开环闭环切换过渡参数-----*/
#define OPEN2CLOSE_RUN_COV_TIME_M0 (30) /* 开环闭环切换过渡时间: 单位: ms 30 */
#define OPEN2CLOSE_RUN_CURRENT_RAMP_M0 (0.05) /* 开环闭环切换过渡内, D,Q轴电流变化斜率 */
/* 当前未用, 预留 */

#define ID_START_M0 (1.0) /* 单位: A, D轴电流设定值 */
#define IQ_START_M0 (1.0) /* 单位: A, Q轴电流设定值 */
```

观测器 PLL 参数及滤波时间设置

```
/* -----观测器PLL PI参数----- */
#define PLL_KP_GAIN_M0 (16 * 32) /* PLL_Kp 估算器Kp 50 16 扩大32 */
#define PLL_KI_GAIN_M0 (8 * 256) /* PLL_Ki 估算器Ki 10 4 扩大25 */
#define PLL_K_FREQ0_M0 (50.0) /* 单位: PLL_K 估算器频率0 */

#define PLL_KP_RUN_M0 (20 * 32) /* 单位: PLL_Kp 估算器Kp1 */
#define PLL_KI_RUN_M0 (8 * 256) /* 单位: PLL_Ki 估算器Ki1 */
#define PLL_K_FREQ1_M0 (70.0) /* 单位: PLL_K 估算器频率1 */

#define THETA_FIL_TIME_M0 (2.0) /* 单位:ms, 滤波时间 */
#define SPEED_FIL_TIME_M0 (6.0) /* 单位:ms, 滤波时间 */
```

8.2 电流环参数配置

任务执行在 AD 采样中断或 MCPWM 中断中，执行频率取决于 PWM 载波频率设置。

当前代码中设置为：

```
static inline void RegMotorCurrentDQInline(STR_MotorFocElement *tS_pFocElement)
{
    if(!gS_FocObjM[tS_pFocElement->m_bObj_ID].bObserWork)
    {
        #if (VF_START_FUNCTION_ENABLE == FUNCTION_ON)
        if(!getCurLoopCloseStatusFromOB(tS_pFocElement->m_bObj_ID)) //电流环开环
        {
            tS_pFocElement->mAcrD.nError = 0;
            tS_pFocElement->mAcrQ.nError = 0;
        }
        else //电流环闭环
        {
            #endif
            tS_pFocElement->mAcrQ.nError = (s32)tS_pFocElement->mStatCurDQCmd.nAxisQ - tS_pFocElement->mAcrQ.nError;
            tS_pFocElement->mAcrD.nError = (s32)tS_pFocElement->mStatCurDQCmd.nAxisD - tS_pFocElement->mAcrD.nError;
            tS_pFocElement->mAcrOutDQ.nAxisQ = CurrentPIRegulator(&tS_pFocElement->mAcrQ); /* Q轴 */
            tS_pFocElement->mAcrOutDQ.nAxisD = CurrentPIRegulator(&tS_pFocElement->mAcrD); /* D轴 */
        }
        // 电压极限圆控制，内部包含自动过调制功能
        CalcVolCircle(tS_pFocElement->m_bObj_ID, &tS_pFocElement->mAcrOutDQ, &tS_pFocElement->mStatVolAB);
    }
}
```

电流环与观测器互斥执行，同为 PWM 载波的一半。如需提高电流环的控制响应能力，可以注释掉第一个框内代码，但中断执行时间加长。

强制执行第二段代码，即将偏差设置为 0 时并启用 VF 控制时，即电流环 VF 控制模式。

设置电流环控制参数

```
/* -----电流环参数----- */
#define IQ_SET_M0 (1.0) /* 单位: A IqRef, Iq给定值 */

#define VQMAX_M0 (0.8) /* Q轴最大输出限制 */
#define VQMIN_M0 (-0.8) /* Q轴最小输出限制 */

#define VDMAX_M0 (0.8) /* D轴最大输出限制 */
#define VDMIN_M0 (-0.8) /* D轴最小输出限制 */

#define P_CURRENT_KP_M0 (0) /* 电流环Kp, 实际运用的Kp会根据这个值和电 */
/* 0 --- 自动计算参数 非0 --- 设定 */
#define P_CURRENT_KI_M0 (1500) /* 电流环Ki, 实际运用的Ki会根据这个值 */
```

电流环输出最大值为标么值，代表实际电压为 0.8*Umax
根据母线电压及 Umax 电压范围进行合适的设置
也可以在程序中动态调整，如在下图电流环 PI 控制函数中添加更速母线电压值波动的限制值，对电流环变量 wLowerLimitOutput、wUpperLimitOutput 动

态设定。

`gS_FocElementObjM[Obid]. mAcrD. wUpperLimitOutput`

`gS_FocElementObjM[Obid]. mAcrD. wLowerLimitOutput`

等变量赋值，最大调制电压可以根据 `getVdcCirCleLim（）` 和调制系数进行计算。

```
static inline void RegMotorCurrentDQInline(STR_MotorFocElement *tS_pFocElement)
{
    if(!gS_FocObjM[tS_pFocElement->m_bObj_ID].bObserWork)
    {
        #if (VF_START_FUNCTION_ENABLE == FUNCTION_ON)
        if(!getCurLoopCloseStatusFromOB(tS_pFocElement->m_bObj_ID)) //电流环开环
        {
            tS_pFocElement->mAcrD.nError = 0;
            tS_pFocElement->mAcrQ.nError = 0;
        }
        else //电流环闭环
        #endif
        {
            tS_pFocElement->mAcrQ.nError = (s32)tS_pFocElement->mStatCurDQCmd.nAxisQ - tS_pFocElement->mAcrQ.nError;
            tS_pFocElement->mAcrD.nError = (s32)tS_pFocElement->mStatCurDQCmd.nAxisD - tS_pFocElement->mAcrD.nError;
        }
        tS_pFocElement->mAcrOutDQ.nAxisQ = CurrentPIRegulator(&tS_pFocElement->mAcrQ);
        tS_pFocElement->mAcrOutDQ.nAxisD = CurrentPIRegulator(&tS_pFocElement->mAcrD);
    }

    // 电压极限圆控制，内部包含自动过调制功能
    CalcVolCircle(tS_pFocElement->m_bObj_ID,&tS_pFocElement->mAcrOutDQ,&tS_pFocElement->mStatCurDQCmd);
}
```

8.3 速度环参数配置

任务执行在 1ms 时间片中。

```
/*-----速度环参数-----*/
#define POWER_LIMIT_VALUE_M0 (10.0) /* 单位: W 限制功率的大小 */
#define POWER_LIMIT_TIME_M0 (5) /* 单位: 速度环周期, 限功率计算 */
#define POWER_LIMIT_SPEED_M0 (150) /* 单位: Hz 限功率转速给定, 根据 */

#define SPEED_LOOP_CNTR_M0 (0) /* 单位: ms 速度环路计算周期 */

#define STATE04_WAITE_TIME_M0 (100) /* Unit: ms 速度变量初始化时间100 */

#define P_ASR_KP_M0 (6000) /* 速度环Kp */
#define P_ASR_KI_M0 (4000) /* 速度环Ki */

#define IQMAX_M0 (2.0) /* 单位:A, 速度环输出最大值 */
#define IQMIN_M0 (-2.0) /* 单位:A, 速度环输出最小值 */

#define SPEED_RUN_ACC_M0 (50.00) /* 单位 Hz/s 速度加速调整值 */
#define SPEED_RUN_DEC_M0 (50.00) /* 单位 Hz/s 速度减速调整值 */
```

最大输出电流配置, IQMAX_M0/IQMIN_M0 需小于电流采样量程, 并在功率器件等的通流能力范围之内。

速度环 PI 参数, 当启用下面代码时, 自动计算

```
/*-----速度环参数初始化-----*/
#if (DRV0_CLOSE_LOOP == SPEED_LOOP)
{
    pObj->m_pMotorSpeed->mSpeedPI.KP = pDrvCfgPara->mS_FBSpdLoop.PASRKp; //速度环
    pObj->m_pMotorSpeed->mSpeedPI.KI = pDrvCfgPara->mS_FBSpdLoop.PASRKi; //速度环
    pObj->m_pMotorSpeed->mSpeedPI.wIntegral = 0;
    pObj->m_pMotorSpeed->mSpeedPI.wUpperLimitOutput = App2CoreCurTrans(pAppToCore,p
    pObj->m_pMotorSpeed->mSpeedPI.wLowerLimitOutput = App2CoreCurTrans(pAppToCore,p
    pObj->m_pMotorSpeed->mSpeedPI.wInError = 0;

    #if 0
    s32 t_wImpKp;
    s32 t_wImpKi;
    t_nImp = pDrvCfgPara->mS_FBSlvcCfg0.m_wSpeedFilTime + 25;
    //自动调节速度环PI功能
    SpeedPIAutoTunning(pObj->mg_nMActrlProcID,
        5000,
        t_nImp,
        &t_wImpKp,
        &t_wImpKi);

    pObj->m_pMotorSpeed->mSpeedPI.KP = sat(t_wImpKp,0,32767); // PI参数限幅
    pObj->m_pMotorSpeed->mSpeedPI.KI = sat(t_wImpKi,0,32767);
    #endif
}
```

未启用时, 由 P_ASR_KP_M0/P_ASR_KI_M0 配置参数决定。

根据实际应用调整 PI 参数, PI 参数受电机的惯量、最大输出电流、最大输出扭矩等电机参数影响。

速度环滤波时间

```
#define SPEED_FIL_TIME_M0 (6.0) /* 单位:ms, 滤波时间 */
```

动态响应高的滤波时间设置为 6ms, 动态性应低的滤波时间可是适当加长, 例如 10ms、20ms。

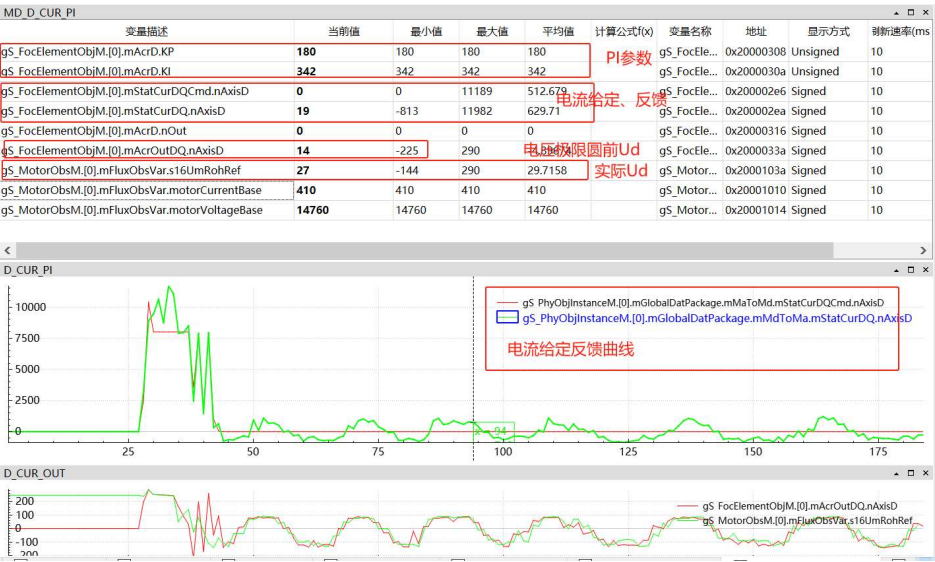
程序调试及应用手册

速度环输出电流指令滤波设定，一般默认 2ms，如有必要可以调整滤波时间。

```
#define D_CURRENT_REF_FIL_TIME_M0      (2.0)      /* 单位: ms D轴电流指令滤波时间常数 */
#define Q_CURRENT_REF_FIL_TIME_M0      (2.0)      /* 单位: ms Q轴电流指令滤波时间常数 */
```

8.4 电流环调试及参数查看

d 轴电流环变量查看



q 轴电流环变量查看



其他变量可以添加到 LksScope 中查看，实时变量用 RTT 查看。

所有 FOC 的变量信息在 gS_FocElementObjM[]数据结构中均可查看，库中的变量提供了查询函数，具体见库函数文档

```
STR MotorFoc      gS_FocObjM[MAX_DRV_MODULE_USED];           // FOC 控制对象定义
STR MotorFocElement gS_FocElementObjM[MAX_DRV_MODULE_USED];   // FOC 变量定义
u8                g_buMdPwMondelay[MAX_DRV_MODULE_USED];      // PWM ON delay 计数
```

8.5 速度环调试及参数查看

其他变量可以添加到 LksScope 中查看，实时变量用 RTT 查看。



8.6 观测器参数查看

观测器参数查看

gS_DcBusVolM.[0].nBusVolt_PerUnit	6301	0	0	6298.74
gS_DcBusVolM.[0].nVoltageCircleLim	5325	5309	5341	5326.07
gS_MotorObsM.[0].mFluxObsVar.wOb_F1	28881	28881	28881	28881
gS_MotorObsM.[0].mFluxObsVar.wOb_F2	6285	6285	6285	6285
gS_MotorObsM.[0].mFluxObsVar.wOb_F3	19692	19692	19692	19692
gS_MotorObsM.[0].mFluxObsVar.bSftN_F1	0	0	0	0
gS_MotorObsM.[0].mFluxObsVar.bSftN_F2	13	13	13	13
gS_MotorObsM.[0].mFluxObsVar.bSftN_F3	11	11	11	11
gS_MotorObsM.[0].mFluxObsVar.nProductPLL	10919	10919	10919	10919
gS_MotorObsM.[0].mFluxObsVar.nOpenLoopDppKp	21839	21839	21839	21839

VF 控制参数

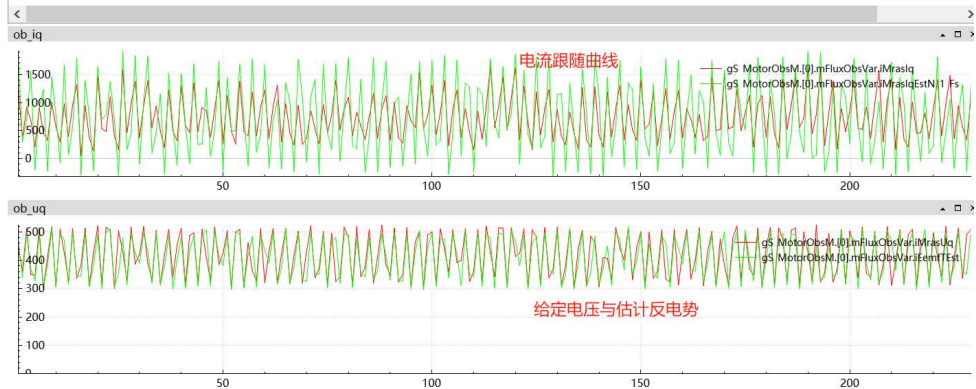
gS_VectorVF.[0].EmfFactorQn	12	12	12	12
gS_VectorVF.[0].EmfFactor	4058	4058	4058	4058
gS_VectorVF.[0].LdQn	12	12	12	12
gS_VectorVF.[0].LdFactor	223	223	223	223
gS_VectorVF.[0].LqQn	12	12	12	12
gS_VectorVF.[0].LqFactor	326	326	326	326
gS_VectorVF.[0].RsQn	12	12	12	12
gS_VectorVF.[0].Rs	50	50	50	50

程序调试及应用手册

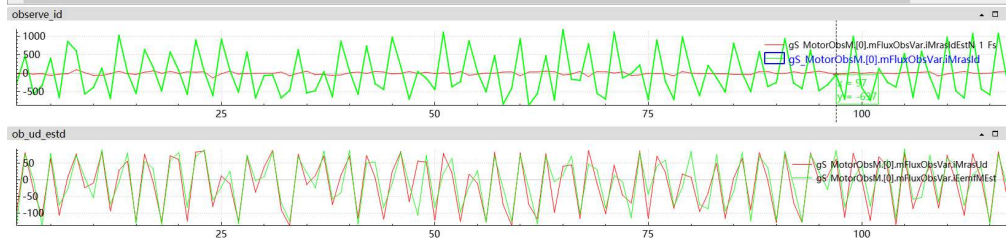
数据查看：

观测器

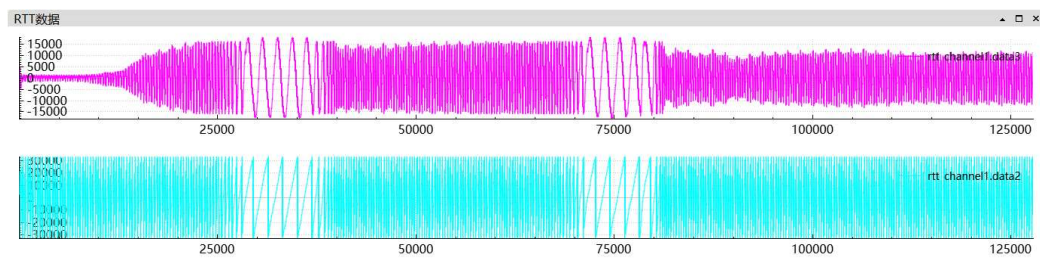
变量描述	当前值	最小值	最大值	平均值	计算公式(x)	变量名称	地址	显示方式	刷新速
gS_MotorObsM[0].mFluxObsVar.iMrasUq	511	300	524	418.144	给定电压	gS_Motor...	0x2000104a	Signed	10
gS_MotorObsM[0].mFluxObsVar.iEmfTEst	502	297	516	409.559	估计反电势	gS_Motor...	0x20001054	Signed	10
gS_MotorObsM[0].mFluxObsVar.iMrasIq	1068	57	1661	747.738	给定电流	gS_Motor...	0x2000104e	Signed	10
gS_MotorObsM[0].mFluxObsVar.iMrasIqEstN_1_Fs	1373	-315	1912	734.175	估计电流	gS_Motor...	0x20001060	Signed	10



变量描述	当前值	最小值	最大值	平均值	计算公式(x)	变量名称	地址
gS_MotorObsM[0].mFluxObsVar.iMrasUd	-49	-137	88	-11.4017		gS_MotorObsM[0].mFluxObsVar.iMrasUd	0x
gS_MotorObsM[0].mFluxObsVar.iMrasUq	511	304	523	421.752		gS_MotorObsM[0].mFluxObsVar.iMrasUq	0x
gS_MotorObsM[0].mFluxObsVar.iEmfMEst	-15	-137	93	-6.93162		gS_MotorObsM[0].mFluxObsVar.iEmfMEst	0x
gS_MotorObsM[0].mFluxObsVar.iEmfTEst	506	293	514	410.188		gS_MotorObsM[0].mFluxObsVar.iEmfTEst	0x
gS_MotorObsM[0].mFluxObsVar.iMrasIdEstN_1_Fs	48	-124	97	2.21368		gS_MotorObsM[0].mFluxObsVar.iMrasIdEstN_1_Fs	0x
gS_MotorObsM[0].mFluxObsVar.iMrasIqEstN_1_Fs	816	-344	1948	735.333		gS_MotorObsM[0].mFluxObsVar.iMrasIqEstN_1_Fs	0x
gS_MotorObsM[0].mFluxObsVar.iMrasId	-149	-859	1178	-12.1453		gS_MotorObsM[0].mFluxObsVar.iMrasId	0x
gS_MotorObsM[0].mFluxObsVar.iMrasIq	645	40	1601	735.385		gS_MotorObsM[0].mFluxObsVar.iMrasIq	0x
gS_MotorObsM[0].mFluxObsVar.m_pFluxObsGain->nD_CurrLoopKP	180	180	180	180	观测器参数	gS_MotorObsM[0].mFluxObsVar.m_pFluxObsGain->nD_CurrLo...	0x
gS_MotorObsM[0].mFluxObsVar.m_pFluxObsGain->nD_CurrLoopKI	342	342	342	342		gS_MotorObsM[0].mFluxObsVar.m_pFluxObsGain->nD_CurrLo...	0x
gS_MotorObsM[0].mFluxObsVar.s16UTRohRef	315	300	525	417.846		gS_MotorObsM[0].mFluxObsVar.s16UTRohRef	0x
gS_MotorObsM[0].mFluxObsVar.iMarsWeEst	990	939	1520	1254.76		gS_MotorObsM[0].mFluxObsVar.iMarsWeEst	0x



运行中加载堵转、降低负载，RTT 电流及角度波形



附录

R.1 数据格式

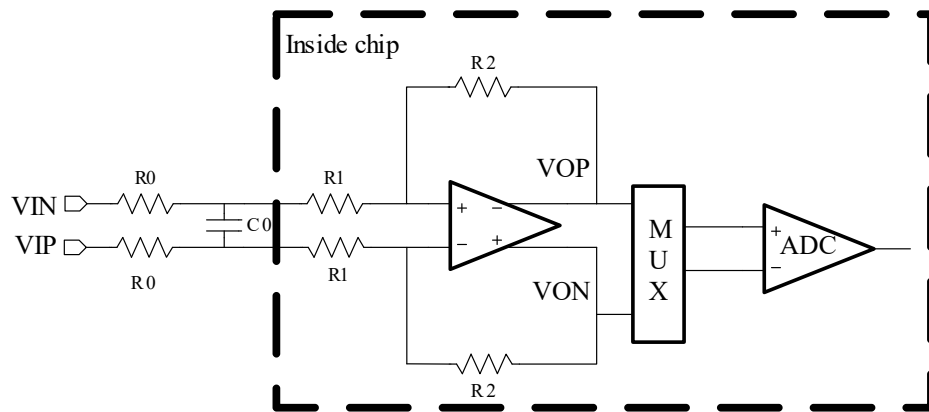
为减少代码空间、提高执行速度，当前程序内部的数据计算采用定点 16 位数据格式，数据格式按照 16 位标么值进行处理。

R.1.1 标么定义

当前的 MCU 内部集成 12 位 AD 运算放大器，转化结果推荐统一使用左对齐方式。

为减少计算量，程序中的数据采用 Q15 格式。

R.1.1.1 电流标么



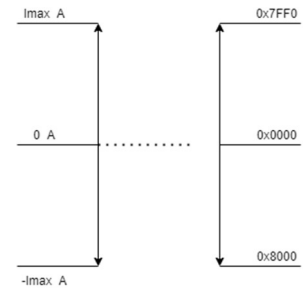
两个 R0 是片外需放置的电阻，阻值必须相等，最终的放大倍数为

$$\text{Gain} = R2 / (R1 + R0)$$

ADC 的输入信号范围最大为 +/-3.6V，反馈电阻 R2:R1 的阻值可通过寄存器 RES_OPAX[1:0]设置，以实现不同的放大倍数。

因此采样电路的最大采样电流 $I_{\text{max}} = 3.6 / (\text{Gain} * R_{\text{shunt}})$
其中 R_{shunt} 为采样电阻阻值。

由于硬件电路的偏置等原因可能会导致差分采样时，整个采样电路能采集到的正向最大采样电流 $I_{\text{max_pos}}$ 和负向最大采样电流 $I_{\text{max_neg}}$ 会有微小的差异，并且在采样结果左移四位后也有略微的数据不同，在允许的误差范围之内，为简便处理，统一表示为 I_{max} 对应 Q15 数据的 0x7FF0，即采用如下的标么格式

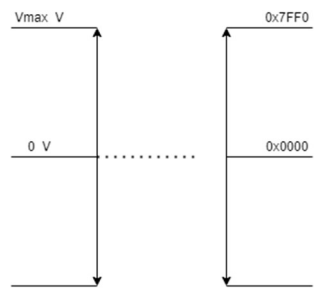


通过采用最大值进行标么后，AD 的采样结果即为 Q15 的标么数据，程序直接读取 AD 的采样结果进行数据处理。

R.1.1.2 电压标么

根据母线电压采样方式，计算最大的电压采样范围

$$V_{\max} = V_{\text{adc}} * (R1 + R2) / R2$$



当反电势采样与母线采样电路不同时，采用各自的标么格式，数据需要转换时进行 AD 采样数据的处理，变换为统一数据格式，程序中需要添加相应的处理。

R.1.1.3 速度标么

基于当前电机运行最高频率，选定合适的最高频率 fmax 作为基准值，即 IQ15(1), 然后进行相应的数据标么处理，最终所有表示的频率数据均为 16 位数据。

R.1.1.4 角度标么

基于电角度，360° 为 65536，进行相应的数据标么处理，角度作为有符号数时表示为：

- 0 ~ 180° : 0 ~ 32767
- 0 ~ -180° : -0 ~ -32768

角度作为无符号数时，0~360° 对应 0 ~ 65536

R.1.2 标么函数

当前的代码分为用户层、电机应用层、电机控制层。

针对当前程序中采用的数据，用户层数据为 User Data，电机应用层数据为 App Data，电机控制层为 Core Data。

其相应的数据关系如下图表格所示。

物理量	User Data	App Data		Core Data	
	实际单位(float)	表示数值	单位刻度	表示数值	单位刻度
电压	1.0V	100	0.01V	$32767 * 100 / (100 * U_{max})$	$32767 / U_{max}$
	U_{max}	$100 * U_{max}$		32767	
电流	1.0A	1000	0.001A	$32767 * 1000 / (1000 * I_{max})$	$32767 / I_{max}$
	I_{max}	$1000 * I_{max}$		32767	
频率	1.0Hz	100	0.01Hz	$32767 * 100 / (100 * f_{max})$	$32767 / f_{max}$
	f_{max}	$100 * f_{max}$		32767	
功率	1.0W	100	0.01W	$32767 * 100 / (100 * P_{max})$	$32767 / P_{max}$
	$P_{max} = U_{max} * I_{max}$	$100 * P_{max}$		32767	

用户层数据即 User Data 采用 float 数据时，代码编译后会引入浮点库，占用较大的 FLASH 空间。

因此，默认用户层采用 App Data 的数据格式进行处理。所有的配置参数对外显示表示为 float 数据，实际程序中应用时通过宏定义或者通过 LKSMotor 转换为定点数进行处理。

App Data 到 Core Data 的转换函数

```
extern s32 App2CoreCurTrans(STR_TransCoefElement* pApp2CoreCur,s32 val);
extern s32 App2CoreFreqTrans(STR_TransCoefElement* pApp2CoreFreq,s32 val);
extern s32 App2CoreVoltTrans(PSTR_TransCoefElement pApp2CoreVolt,s32 val);
```

Core Data 到 App Data 的转化函数

```
extern s32 Core2AppCurTrans(STR_TransCoefElement* pCore2AppCur,s32 val);
extern s32 Core2AppFreqTrans(STR_TransCoefElement* pCore2AppFreq,s32 val);
extern s32 Core2AppVolTrans(STR_TransCoefElement* pCore2AppVol,s32 val);
extern s32 Core2AppPowerTrans(PSTR_TransCoefElement pCore2AppPower,s16 val);
```

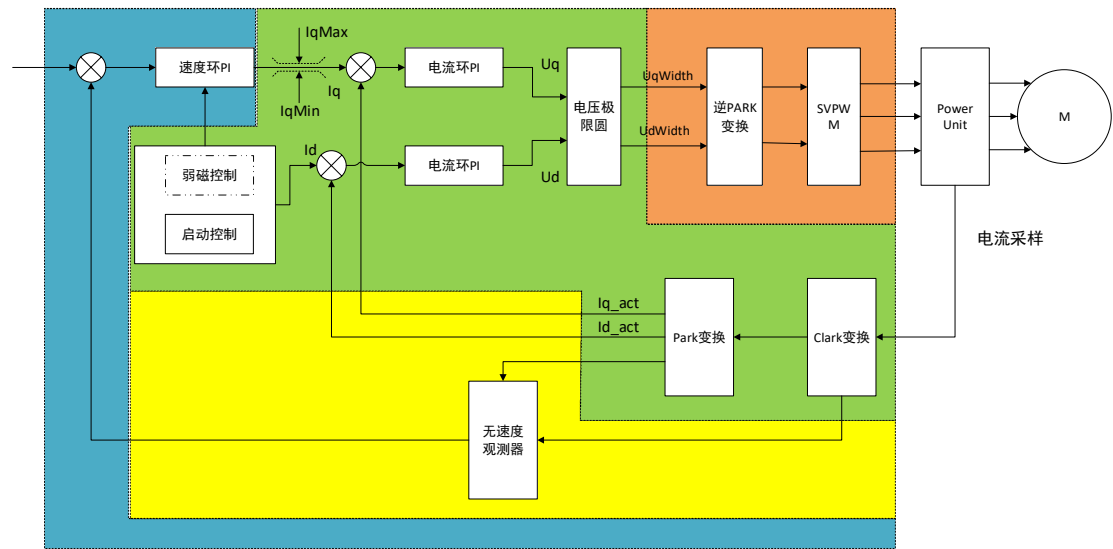
程序中包含预留的浮点数据转换函数接口，用户在接受占用较大 FLASH 空间的前提下可以调用浮点标么转换函数进行数据处理。

预留的 User Data 到 App Data 的浮点转换函数：

```
extern s32 User2AppCurTrans(STR_TransCoefElement* pUser2AppCur,float val);
extern s32 User2AppFreqTrans(STR_TransCoefElement* pUser2App,float val);
extern s32 User2AppVolTrans(PSTR_TransCoefElement pUser2AppVolt,float val);
```

R.1.3 数据表示

在如下的简要控制框图中，用不同的色块对整个控制框图进行了大概的划分。

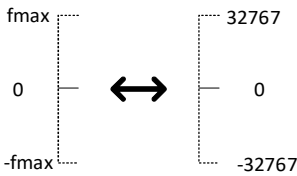


 区域中，

Core Data 数据表示为下面的映射关系，App Data 数据需按照约定的单位刻度进行标定，如 1A 表示为 1000 等。

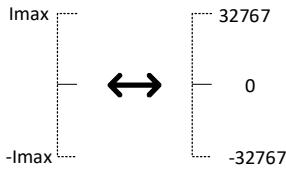
输入量

频率或转速



输出控制量

电流

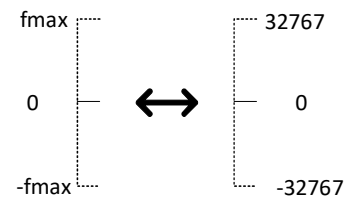


、 区域中，

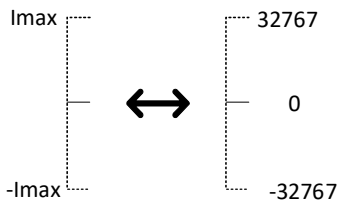
Core Data 数据表示为下面的映射关系，App 数据按照单位刻度进行标定。

输入量

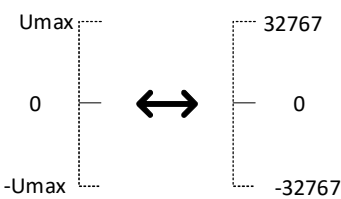
频率或转速



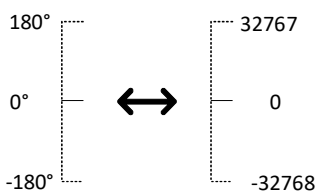
电流



母线电压

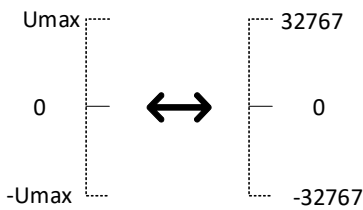


角度



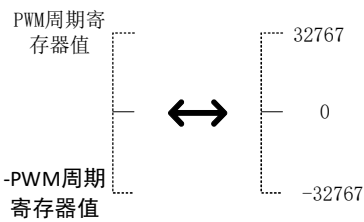
输出量:

dq 电压及占空比



 区域中,

Core Data 数据表示为下面的映射关系

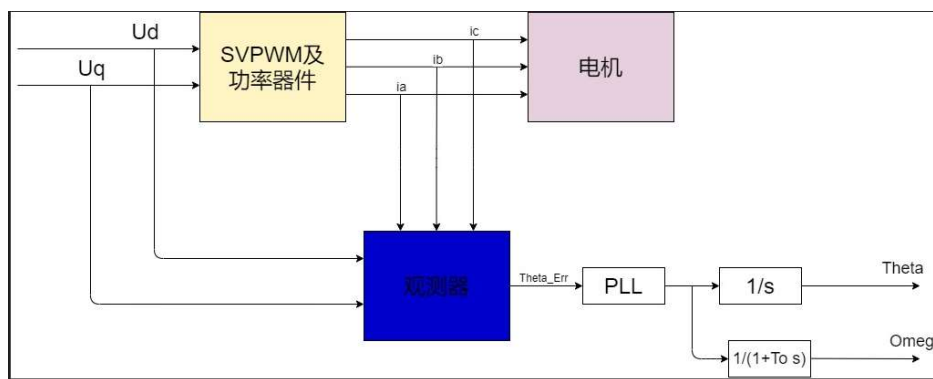


R.2 观测器介绍

FOC 磁场定向，两相同步旋转坐标上，永磁同步电机数学模型

$$\begin{aligned} U_d &= R_s i_d + L_d \frac{di_d}{dt} - \omega L_q i_q \\ U_q &= R_s i_q + L_q \frac{di_q}{dt} + \omega \varphi_f + \omega L_d i_d \\ \varphi_d &= \varphi_f + L_d i_d \\ \varphi_q &= L_q i_q \end{aligned}$$

观测器实现过程框图如下：



根据电机模型采，采用状态观测器观测电机的反电势 E_d 、 E_q ，反正切后求得角度误差 $Theta_{err}$ ，经 PLL 后计算出 FOC 角度 $Theta$ 及电频率 $Omeg$ 。

观测器参数采用与电流环相同的 PI 控制器参数。

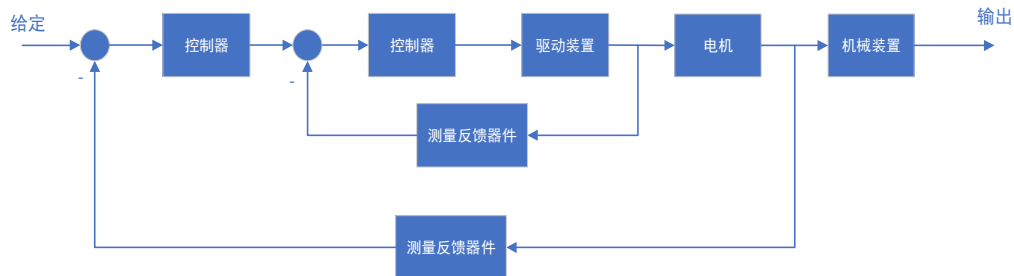
PLL 参数根据角度基准值、频率基准值进行标么化后处理，kp 数据格式 IQ12，ki 数据格式 IQ22

采用 16K 开关频率、基准频率 1333Hz 时，比例积分系数分别为 16*32, 8*256

高速时根据需要增大 PLL 的 Kp/Ki 参数，增强系统的跟随性。

R.3 参数整定

闭环控制系统利用给定与输出产生偏差信号进行 PI 反馈调节。



比例积分调节器（PI 调节器）的输入输出关系为

$$U_{ex} = K_p U_{in} + \frac{1}{\tau} \int_0^t U_{in} dt$$

式中， U_{in} ——PI 调节器的输入， U_{ex} ——PI 调节器的输出。

其传递函数为

$$W_{PI}(s) = K_p + \frac{1}{\tau s} = \frac{K_p \tau s + 1}{\tau s}$$

式中， K_p 、 τ ——PI 调节器的比例放大系数、积分系数

通常采用 PI 调节 $G(s) = K_p + K_i \frac{1}{s}$

式中， K_p 、 K_i ——PI 调节器的比例放大系数、积分系数

程序中应用自动参数计算时，PI 参数自动进行整定。

以下示例中的参数计算，基于当前示例硬件设置：

- 1) 电流采样范围 $I_{\max} = 4.10 \text{ A}$
- 2) 电压采样范围 $U_{\max} = 147.6 \text{ V}$
- 3) 开关频率 $f_{PWM} = 16 \text{ KHz}$

R.3.1 电流环 PI 参数

PI 调节 $G(s) = K_p + K_i \frac{1}{s}$

整定的参数 K_p 为 IQ12 数据格式， K_i 为 IQ16 数据格式：

$$K_p = L * 0.77164$$

$$K_i = R * 0.077164$$

其中

R: 电机电阻, 单位为 0.1 毫欧

L: 电机 dq 轴电感, 单位为 1uH

整定的 PI 参数不合适时可自行进行缩放处理。

电流环 PI 与观测器 PI 参数采用形同的数据。如果需要采用不同的参数时, 自行计算参数用于观测器, 电流环参数手动设定。

R.3.2 速度环 PI 参数

$$\text{PI 调节 } G(s) = K_p + K_i \frac{1}{s}$$

自动整定参数 K_p 为 IQ9 数据格式, K_i 为 IQ14 数据格式:

$$K_p = K_{p_set} * \frac{0.11073 * f_{\max}}{Polar * (T_{on} + 2.5)}$$

$$K_i = \frac{2 * K_p}{(T_{on} + 2.5)}$$

其中 K_{p_set} 代表设定倍数, 与惯量成正比, 与电机电安系数成反比, 1000 代表 1.0 倍。

T_{on} : 速度滤波时间, ms

$polar$: 电机极对数

$f_{\max}$: 频率基准值

整定的 PI 参数范围不合适时可修改 IQ 数据格式, 并进行相应的代码修改。

手动设定速度 PI 参数时, 可根据需要自行设定。